

Google OAuth 2.0 Login Integration

User Authentication &
Communication in FastAPI

Tran Thi Ngoc Nhu - 524H0021

Vo Ngoc Thanh Vy - 524H0040

Nguyen Ly Bao Tran - 524H0130

OUTLINE

Google OAuth 2.0 Login Integration



01

THEORY & OAUTH FLOW

Core concepts and login flow



02

INTEGRATION & APPLICATION

Practical uses and best practices



03

SETUP & HANDS-ON DEMO

Setup and implementation steps

WHAT ARE THESE TECHNOLOGIES?

OAuth 2.0

Authorization framework

Allows the app to access user data with permission.

OpenID Connect

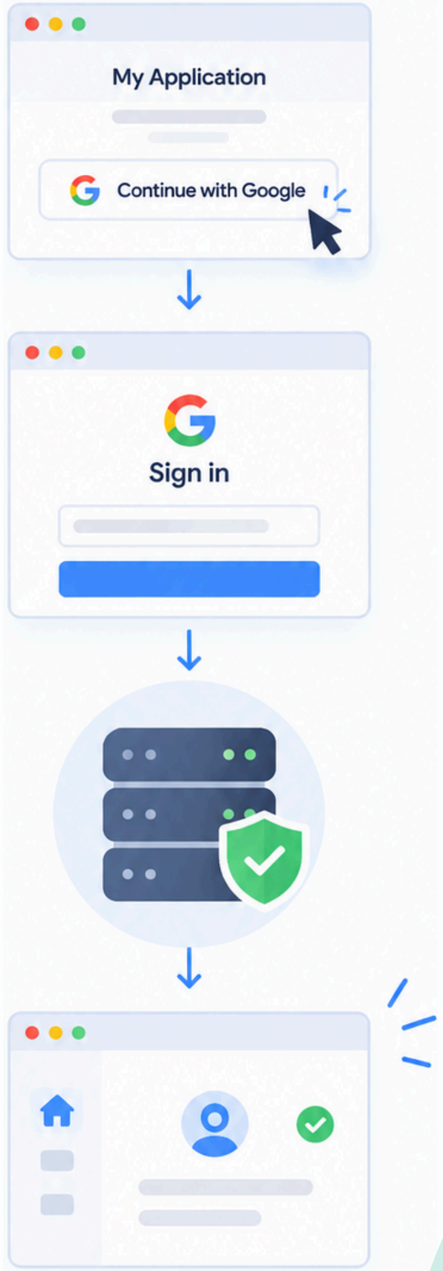
Identity/authentication layer

Provides information about who the user is.

Google Identity

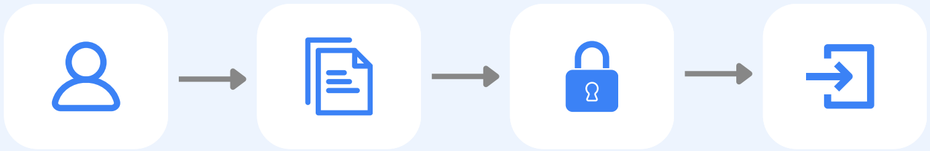
Identity Provider

Verifies the user using their Google account.



WHY GOOGLE LOGIN?

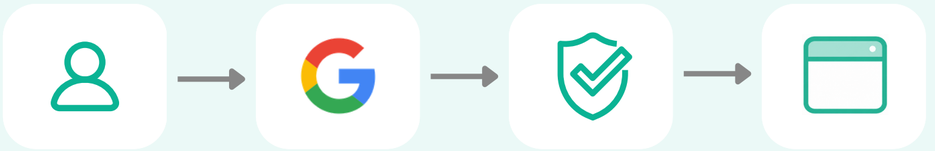
Traditional Login



- Create a new account
- Create & remember a password
- Handle password reset
- Application manages credentials

Application manages auth

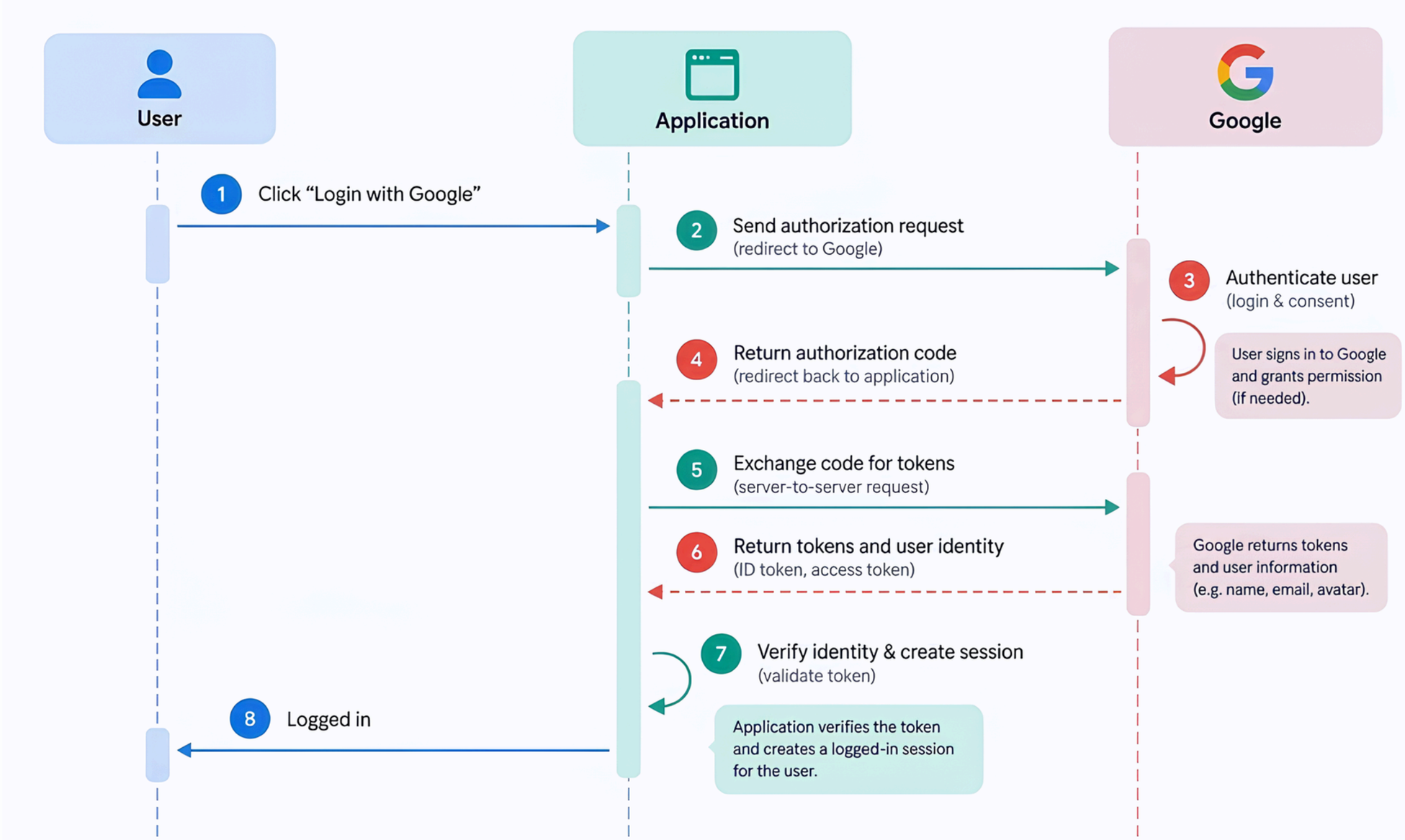
Google Login



- Use an existing Google account
- Sign in with Google
- Google verifies the user
- Application receives verified identity

Reduce authentication complexity and user login friction.

GOOGLE OATH 2.0 LOGIN FLOW



GOOGLE LOGIN IN A FASTAPI MICROSERVICE

User (Client)

Initiates the login request and interacts with the application interface.

FastAPI Microservice

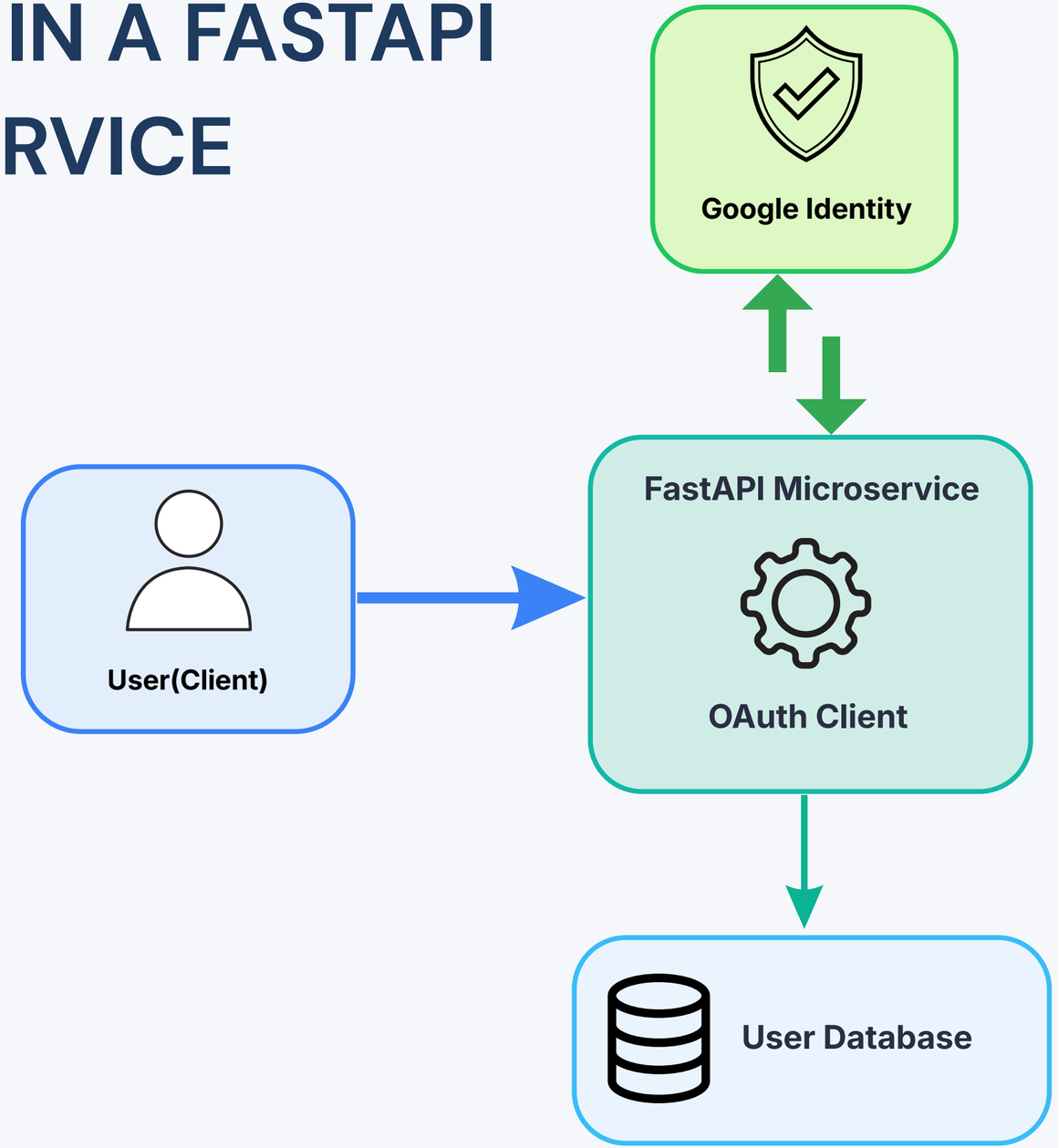
Acts as the Authlib OAuth Client. Uses FastAPI Security to intercept requests and receive user data.

Google (OAuth/OIDC)

Handles the OAuth/OIDC flow, authenticates users, and issues access tokens.

User Database

Stores local records and maps the authenticated Google identity to the internal system user.



WHICH SEMESTER PROJECTS BENEFIT FROM IT?

Sports / Room Booking

Instantly fetch email and profile scopes.
Automate user schedules and booking confirmations.





E-commerce

Skip tedious manual registration.
Enable faster checkout and higher conversion rates.

Internal University Tools

Ensure secure, domain-restricted access.
Allow logins exclusively for @student.domain.edu accounts.





E-Learning & Library Apps

Provide a seamless Single Sign-On (SSO) experience.
Easily track student progress or borrowing history.

SOURCE CODE

What's inside?

- FastAPI Source Code (main.py)
- Environment Setup
- Template (.env.example)
- Step-by-step documentation (README.md)

How to clone?

```
git clone https://github.com/NguyenLyBaoTran/google-oauth-fastapi-demo TestOauth
cd TestOauth
```



Scan to clone the Demo Repository!

FASTAPI SETUP & LIBRARIES

Project Structure

```
ProjectName/
│
├──── .venv/
├──── .env
├──── main.py
└──── requirements.txt
```

Quickstart Command

```
python -m venv .venv
source .venv/bin/activate  # or .venv\Scripts\activate
pip install -r requirements.txt
```

Library	Purpose
fastapi	Builds the web API.
uvicorn	Runs the FastAPI application.
authlib	Handles Google OAuth login.
httpx2	Sends HTTP requests to Google.
python-dotenv	Loads configuration from .env.
itsdangerous	Protects session data with signatures.

ENVIRONMENT VARIABLES SETUP

.env

```
GOOGLE_CLIENT_ID=your_google_client_id

GOOGLE_CLIENT_SECRET=your_google_client_secret

SESSION_SECRET=your_session_secret
```

Security Best Practice: Never hardcode credentials into source code. Always commit a .env.example file without real secrets.

Client ID
Identifies the application

Client Secret
Authenticates the application

Redirect URI
Where Google sends the user back

Session Secret
Secret key for encrypting user session cookie

REGISTERING GOOGLE OAUTH IN FASTAPI

main.py

App Initialization & Config:

```python
load_dotenv()

GOOGLE_CLIENT_ID = os.getenv("GOOGLE_CLIENT_ID")
GOOGLE_CLIENT_SECRET = os.getenv("GOOGLE_CLIENT_SECRET")
SESSION_SECRET = os.getenv("SESSION_SECRET") or "dev-secret-key"

app = FastAPI(
    title="Google OAuth 2.0 Login Demo"
)

app.add_middleware(
    SessionMiddleware,
    secret_key=SESSION_SECRET,
    same_site="lax",
    https_only=False
)
```

Registering Google Provider:

```python
oauth = OAuth()

oauth.register(
    name="google",
    client_id=GOOGLE_CLIENT_ID,
    client_secret=GOOGLE_CLIENT_SECRET,
    server_metadata_url=(
        "https://accounts.google.com/"
        ".well-known/openid-configuration"
    ),

    client_kwargs={
        "scope": "openid email profile"
    }
)
```

IMPLEMENTING AUTH ENDPOINTS

main.py

Login Trigger Endpoint

```python
@app.get("/login/google")
async def login_google(request: Request):
    redirect_uri = request.url_for("google_callback")
    return await oauth.google.authorize_redirect(
        request,
        redirect_uri,
        access_type="offline",
        prompt="consent"
    )
```

Login Initialization Flow

01 User Trigger
User clicks the “Login with Google” button

02 FastAPI Backend
Create Auth URL with redirect_url

03 Google OAuth Page
Redirect user to Google authentication page

IMPLEMENTING AUTH ENDPOINTS

main.py

```python
@app.get("/auth/google/callback")
async def google_callback(request: Request):

    token = await oauth.google.authorize_access_token(request)

    user = token.get("userinfo")

    if not user:
        raise HTTPException(
            status_code=400,
            detail="Cannot get Google user information"
        )

    request.session["user"] = {
        "google_id": user.get("sub"),
        "email": user.get("email"),
        "name": user.get("name"),
        "picture": user.get("picture")
    }

    return {
        "message": "Google OAuth login successful",
        "user": {
            "google_id": user.get("sub"),
            "email": user.get("email"),
            "name": user.get("name"),
            "picture": user.get("picture")
        },
        "token": {
            "access_token": token.get("access_token"),
            "refresh_token": token.get("refresh_token"),
            "token_type": token.get("token_type"),
            "expires_in": token.get("expires_in"),
            "id_token": token.get("id_token")
        }
    }
```

01 Receive Auth Code
Google redirects to the callback with a code

02 Exchange Token
Exchange the code for Access Token & User Info from Google

03 Save User Session
Store user information in encrypted request.session

04 Expose /me Endpoint
Return user profile to the client from the session cookie

IMPLEMENTING AUTH ENDPOINTS

main.py

```python
@app.get("/me")
async def get_current_user(request: Request):
    user = request.session.get("user")

    if not user:
        raise HTTPException(
            status_code=401,
            detail="Not logged in"
        )
    return user

@app.get("/logout")
async def logout(request: Request):

    request.session.clear()

    return {
        "message": "Logout successful"
    }
```

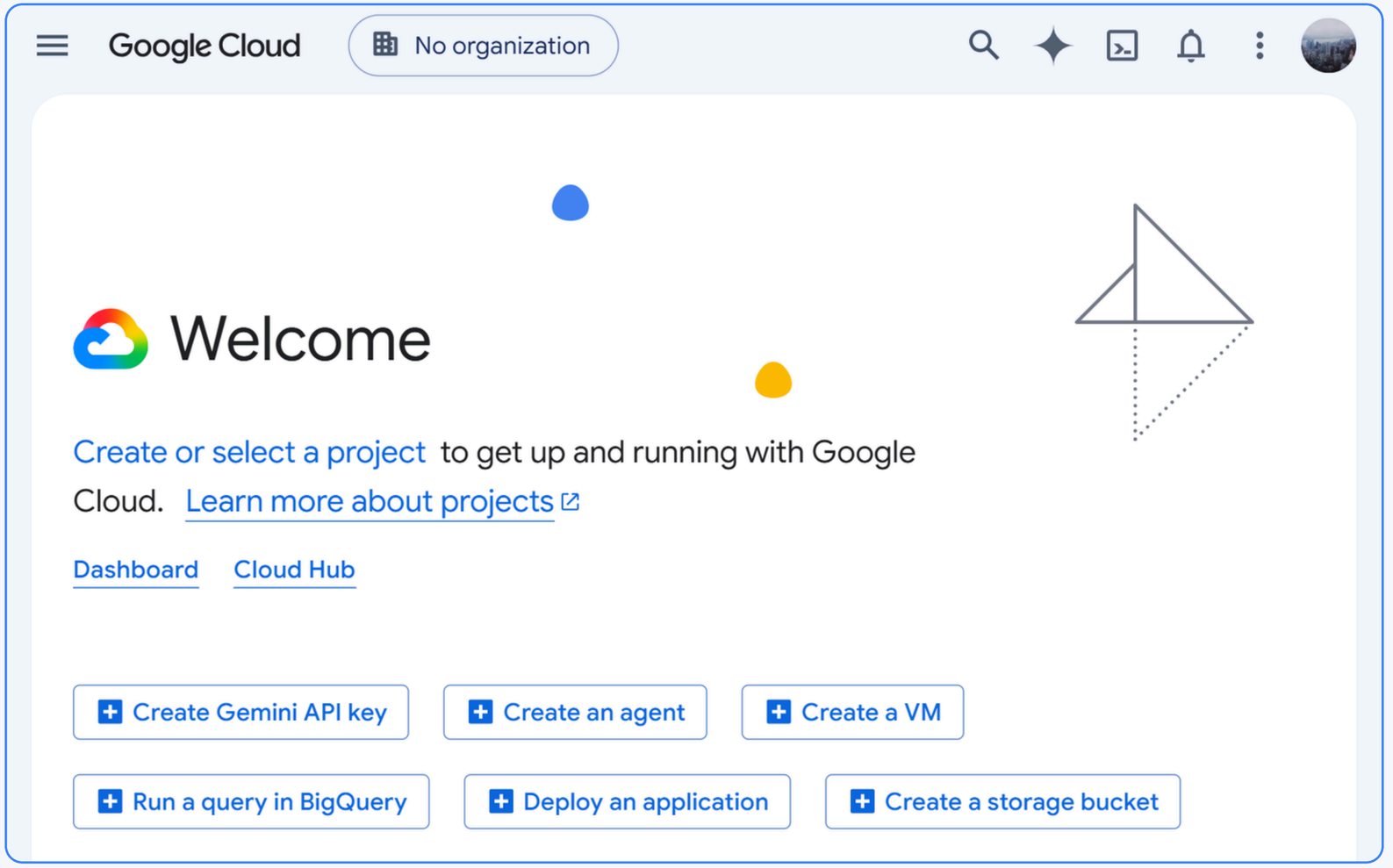
01 Receive Auth Code
Google redirects to the callback with a code

02 Exchange Token
Exchange the code for Access Token & User Info from Google

03 Save User Session
Store user information in encrypted request.session

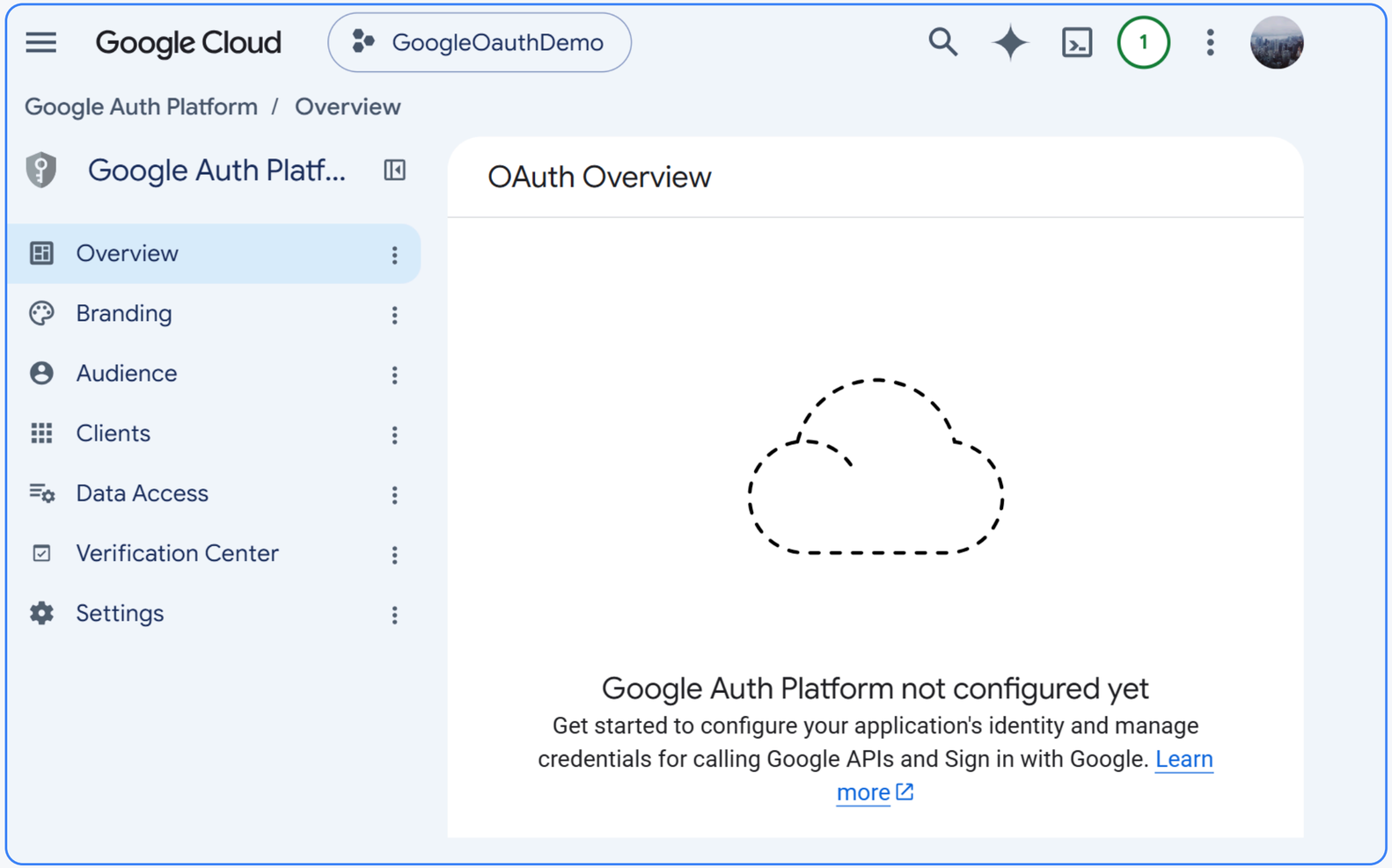
04 Expose /me Endpoint
Return user profile to the client from the session cookie

GOOGLE CLOUD CONSOLE SETUP



Google Cloud Console

GOOGLE AUTH PLATFORM SETUP



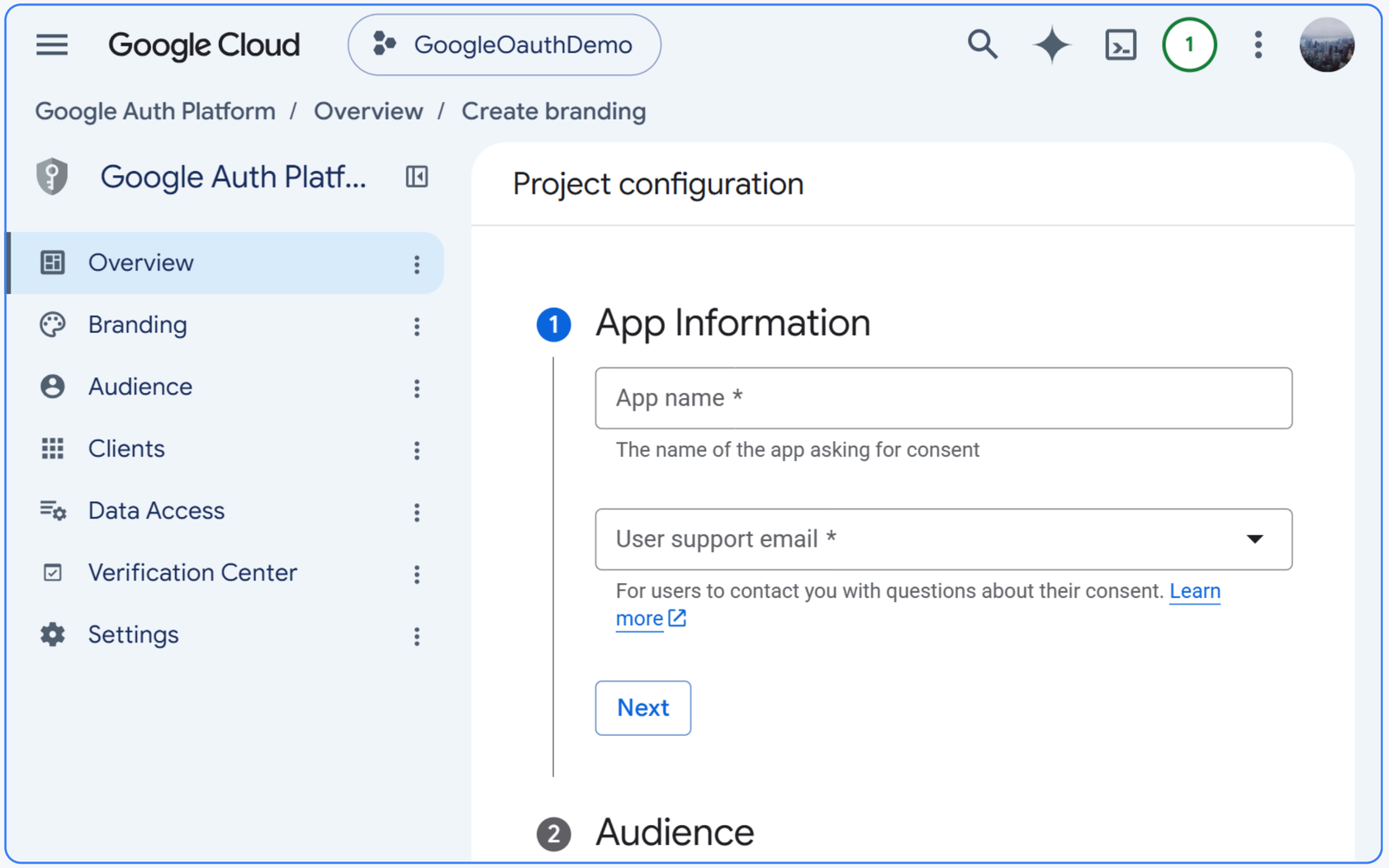
Branding
Defines the application's identity

Audience
Defines who can use the application

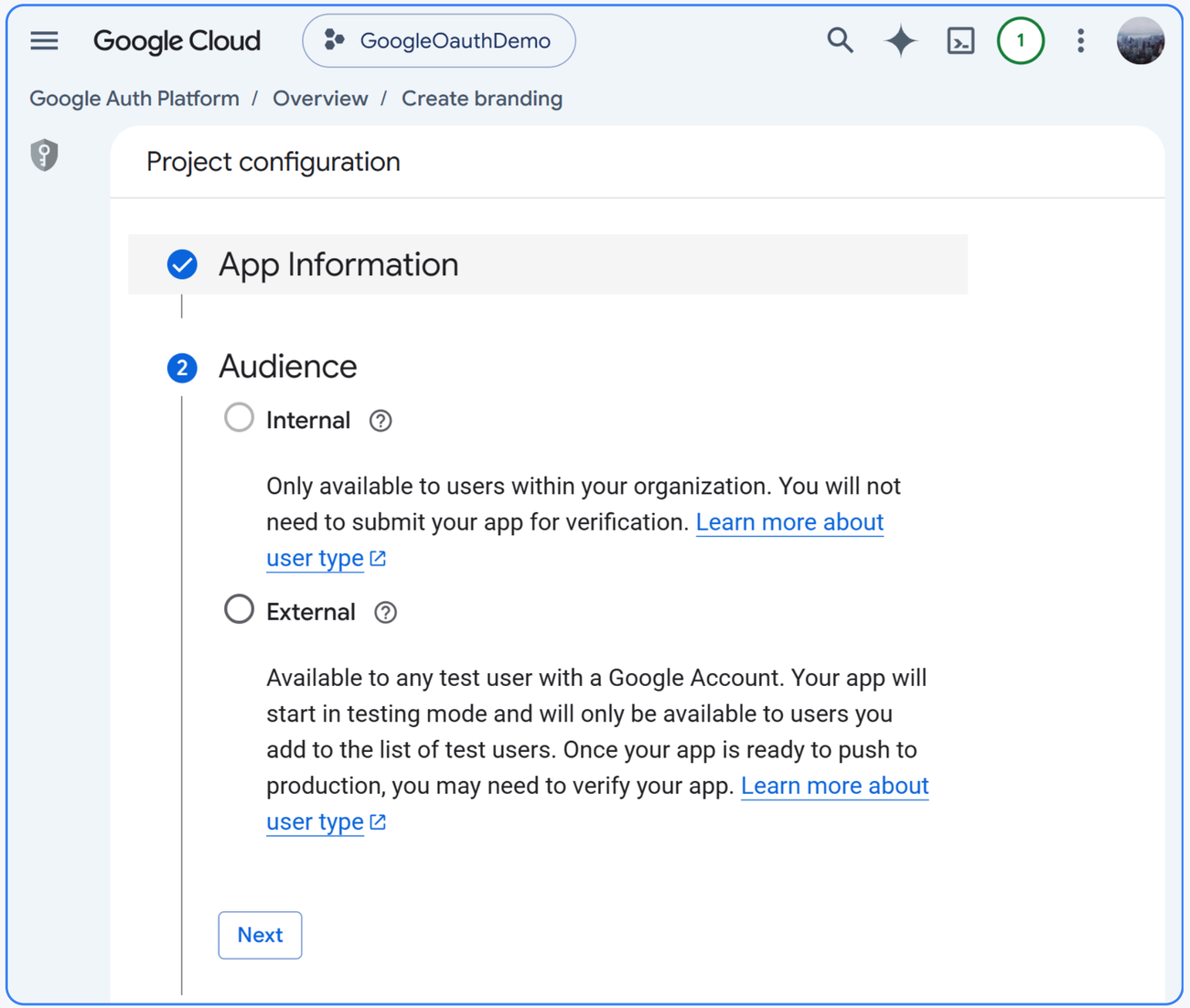
Clients
Identifies the application

Data Access
Defines requested Google permissions

GOOGLE AUTH PLATFORM SETUP



GOOGLE AUTH PLATFORM SETUP



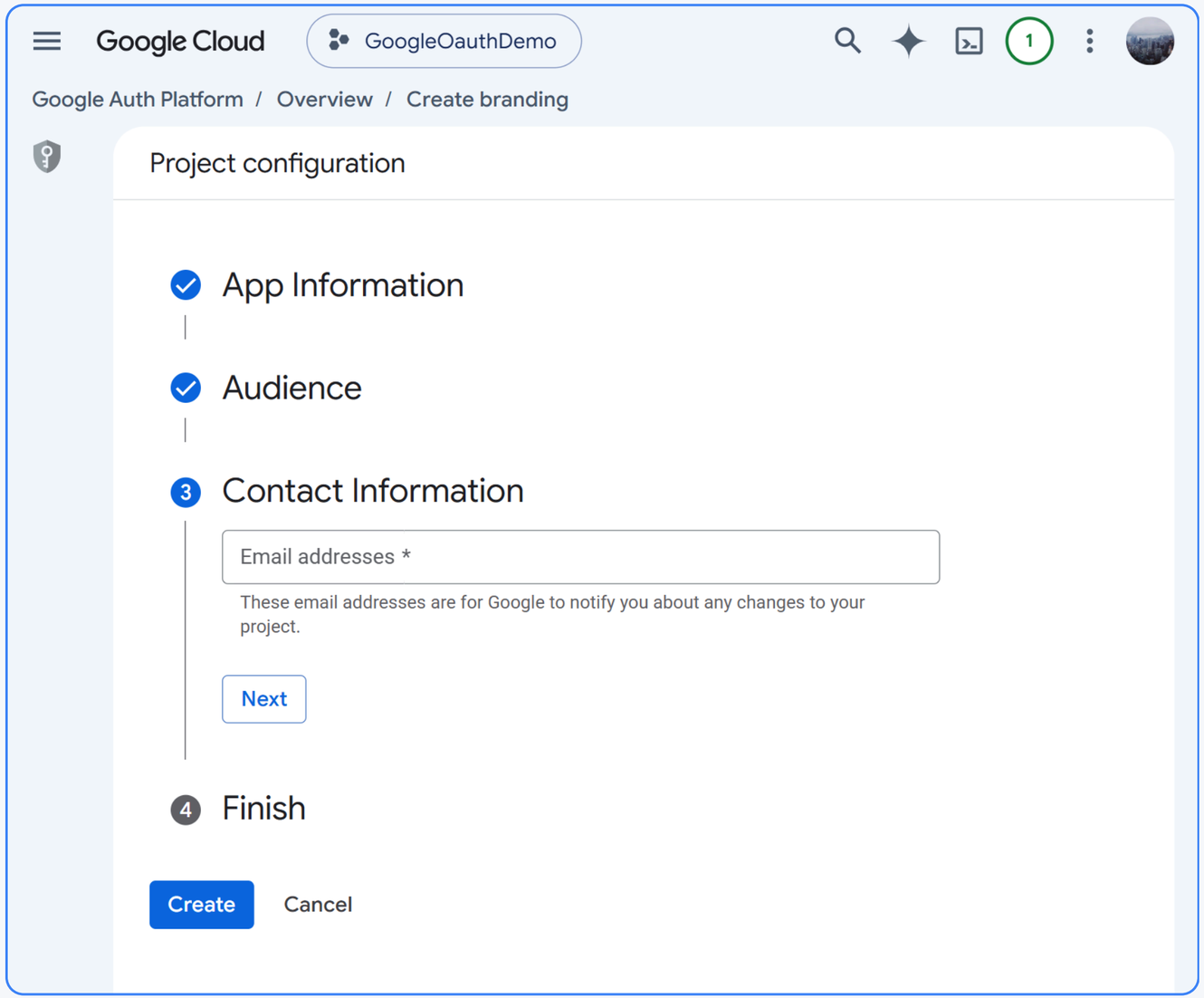
Internal

Only people in your Google Workspace (your company/organization) can log in.

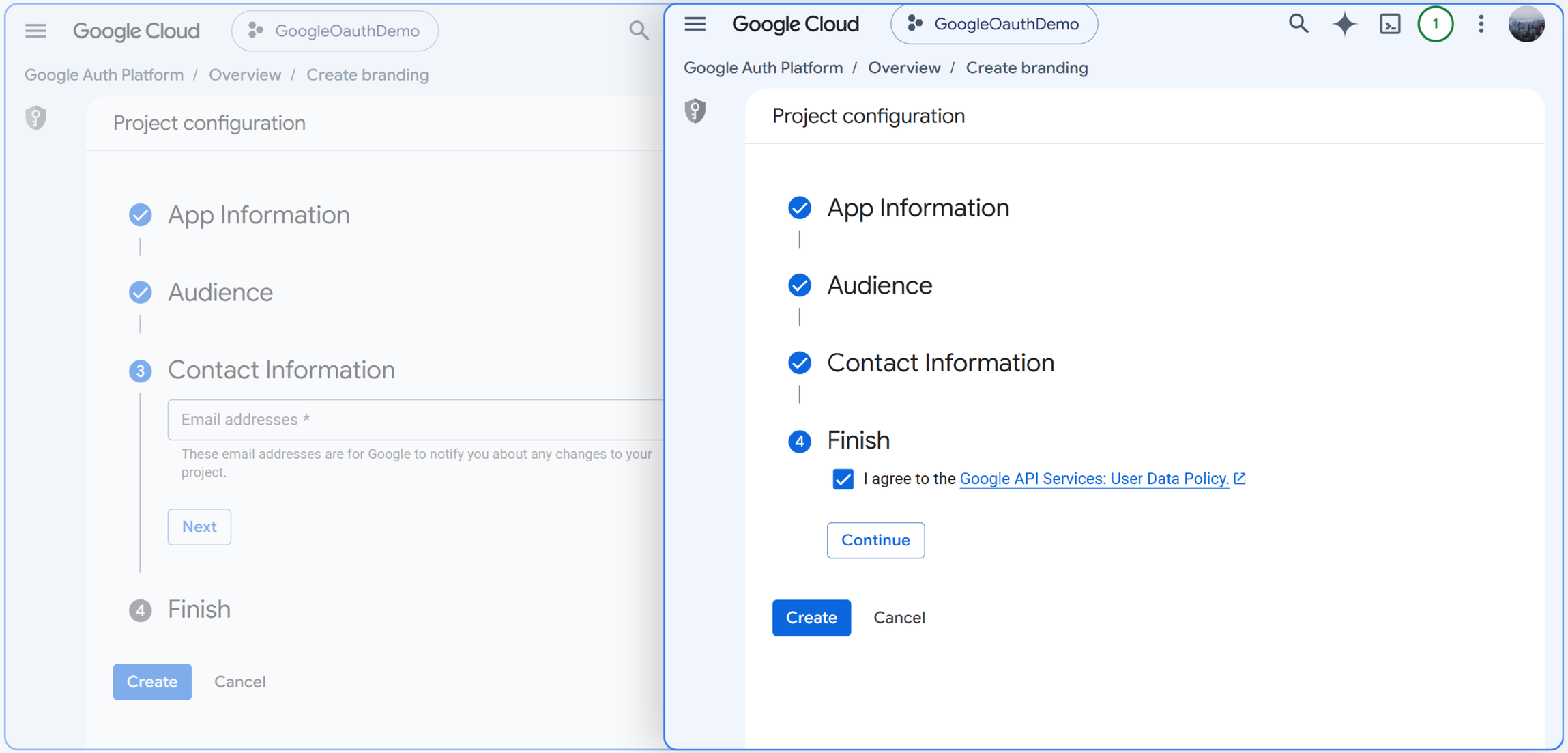
External

Any Google account user worldwide can log in.

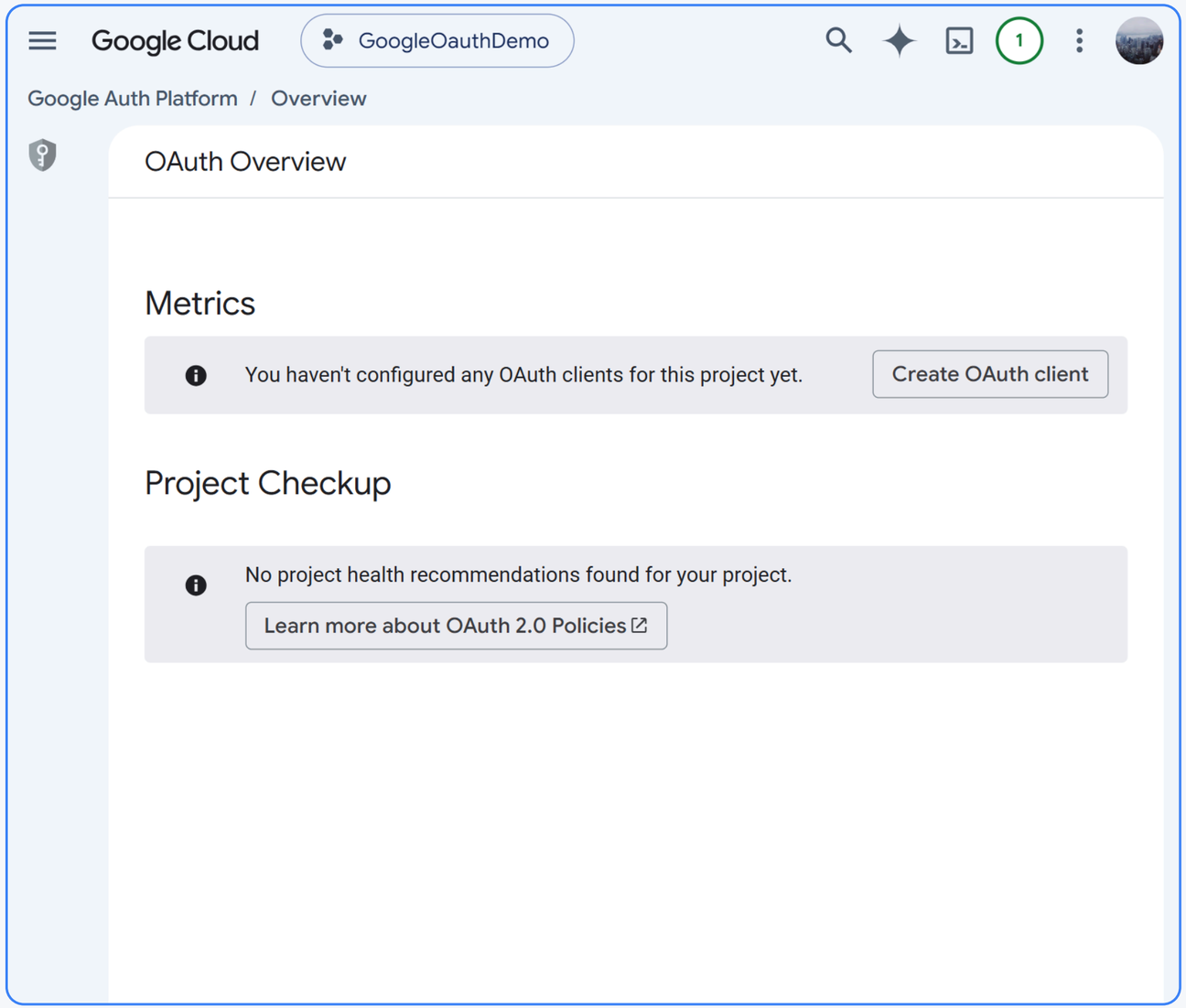
GOOGLE AUTH PLATFORM SETUP



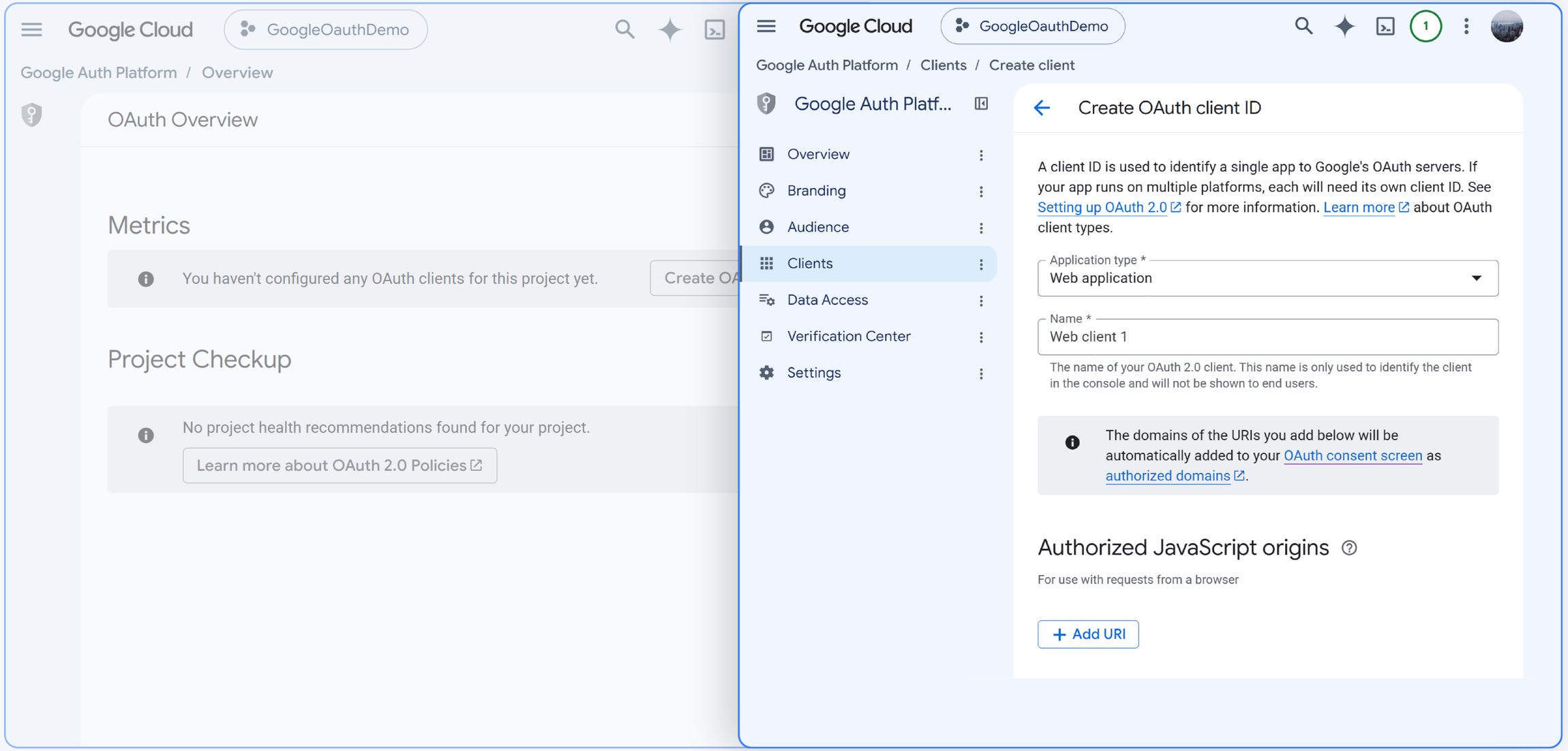
GOOGLE AUTH PLATFORM SETUP



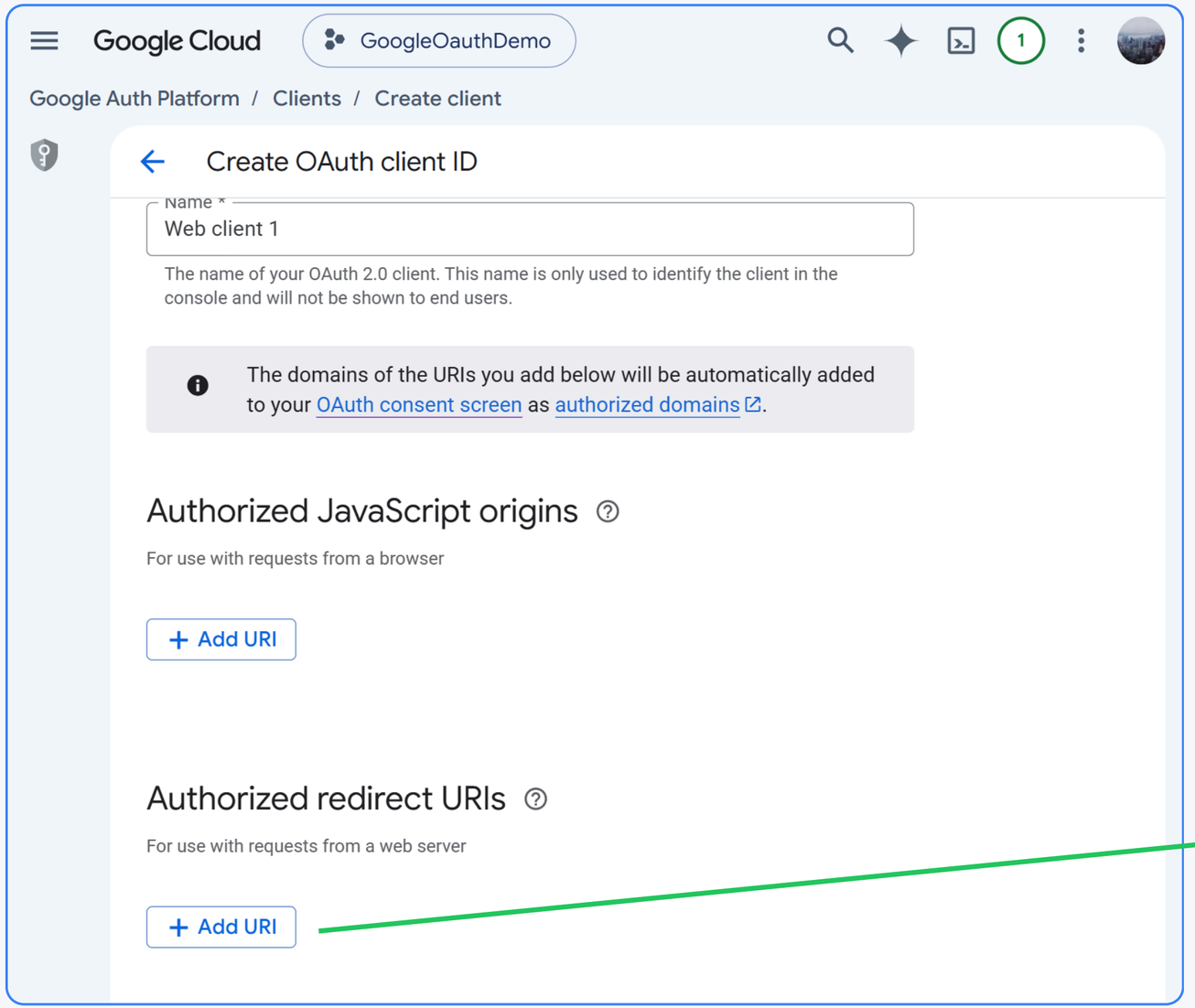
GOOGLE AUTH PLATFORM SETUP



GOOGLE AUTH PLATFORM SETUP



GOOGLE AUTH PLATFORM SETUP



Authorized JavaScript Origins

Defines **WHO** is allowed to launch the sign-in pop-up/request.

Authorized Redirect URIs

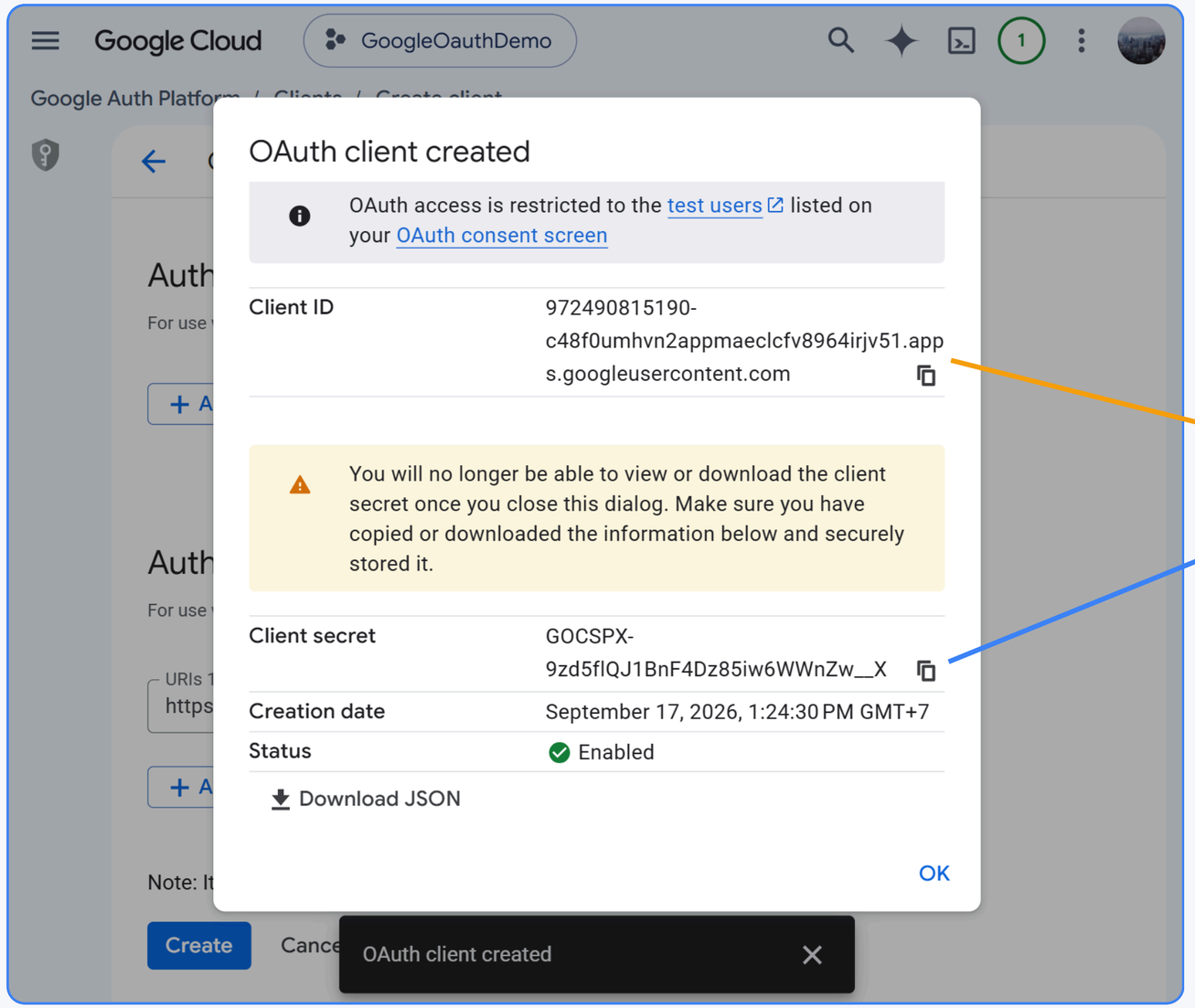
Defines **WHERE** the user and authorization data are sent once sign-in succeeds.

Sample Redirect Endpoint

http://[LOCAL_URL]/auth/google/callback

https://[NGROK_URL]/auth/google/callback

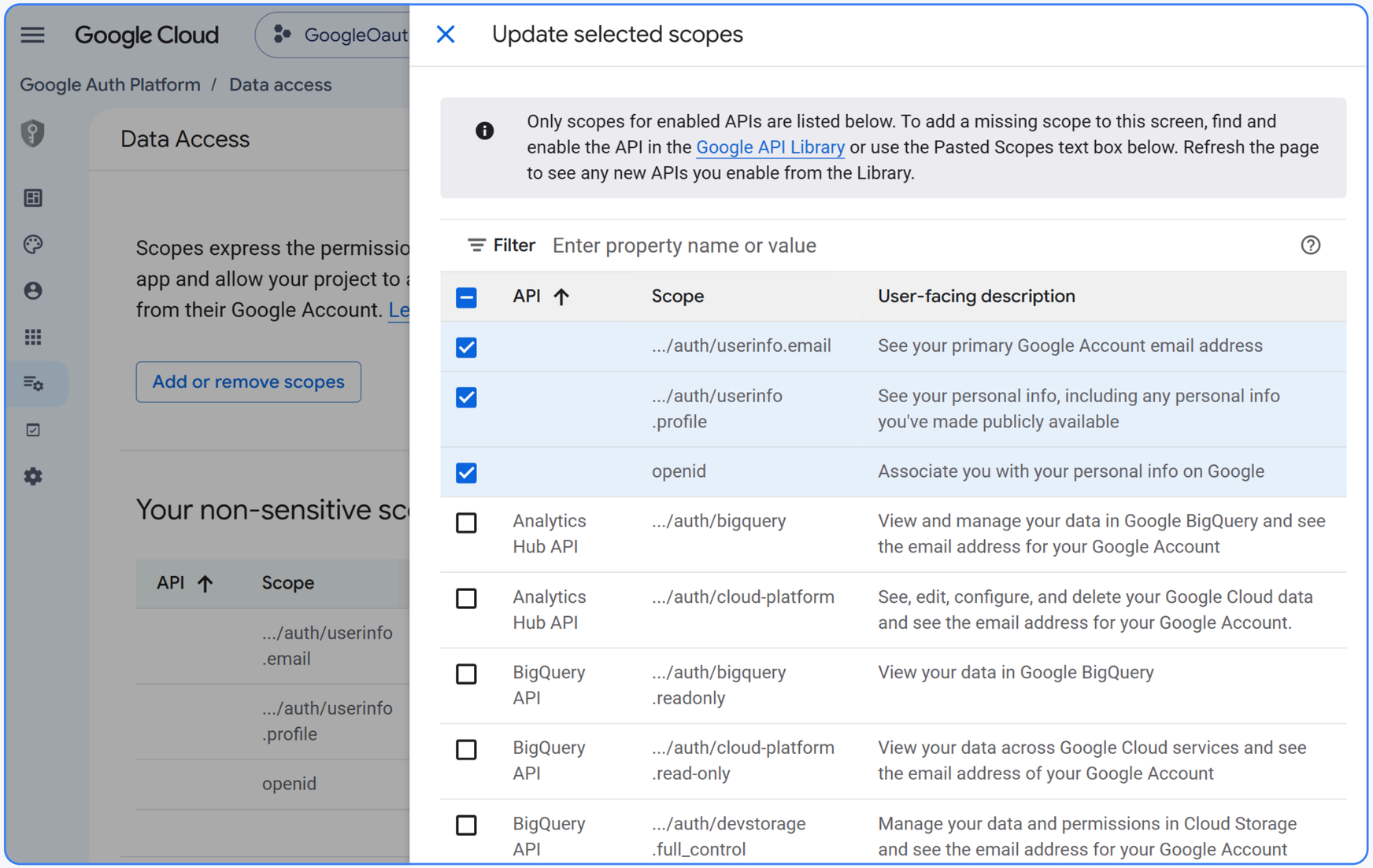
GOOGLE AUTH PLATFORM SETUP



.env

```
1  GOOGLE_CLIENT_ID=your_google_client_id
2
3  GOOGLE_CLIENT_SECRET=your_google_client_secret
```

GOOGLE AUTH PLATFORM SETUP

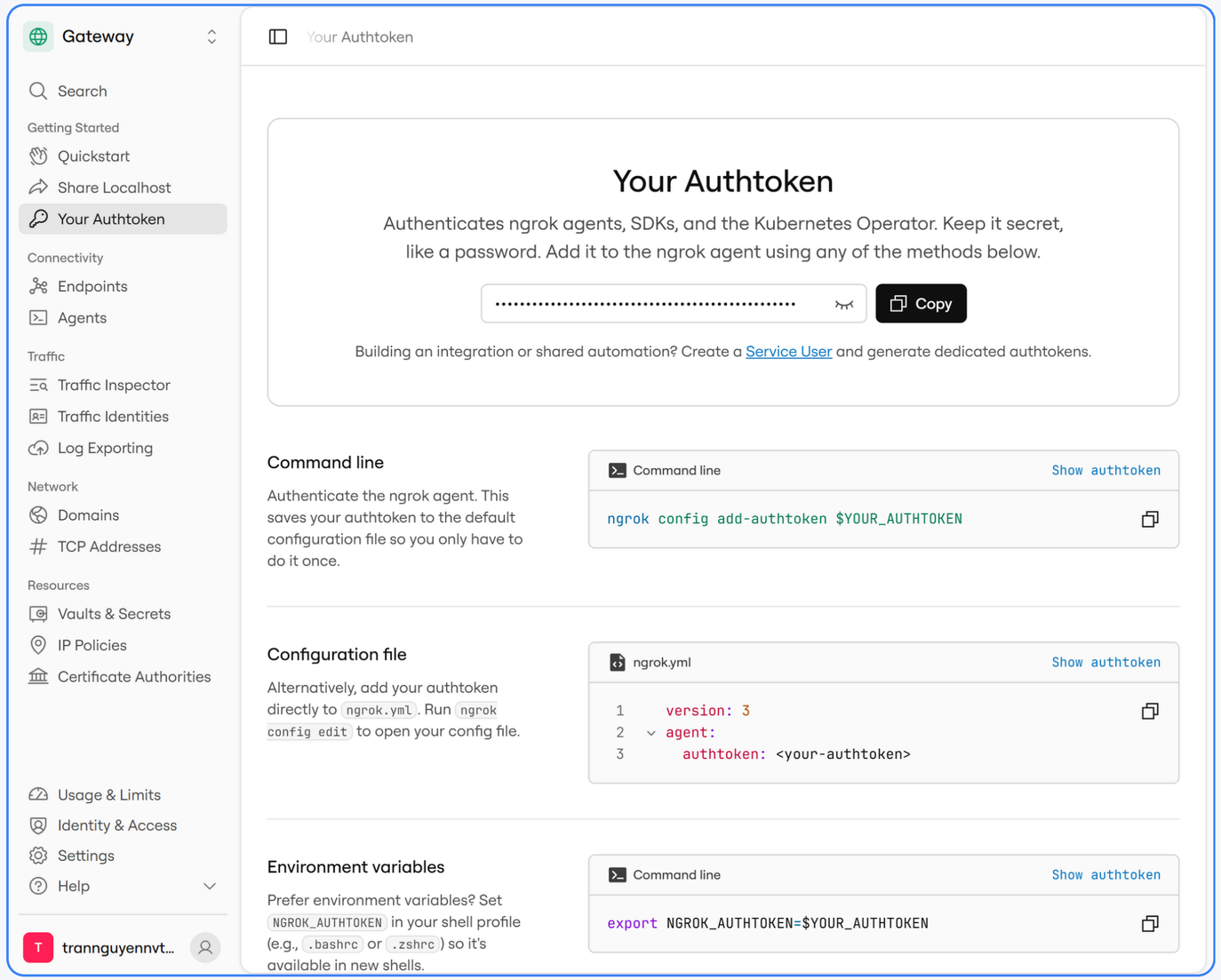


openid
Proves **WHO** the user is.

userinfo.email
Gives you **WHERE** to reach them.

userinfo.profile
Shows you **HOW** to display their info on your UI.

CONFIGURE NGROK



01 Install ngrok CLI

02 Get Authtoken

03 Authenticate Agent

04 Start HTTP Tunnel

05 Obtain Public HTTPS Domain

06 Sync Redirect URI

CONFIGURE NGROK

```
ngrok                                                                    (Ctrl+C to quit)

◆  One gateway for every AI model. https://app.ngrok.ai

Session Status                online
Account                       trannguyennvt09870987@gmail.com (Plan: Free)
Version                       3.39.11-msix-stable
Region                        Asia Pacific (ap)
Latency                       60ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://talcum-alias-swimming.ngrok-free.dev -> http://localhost:8081

Connections                   ttl     opn     rt1     rt5     p50     p90
                              0       0       0.00    0.00    0.00    0.00
```

Terminal 1 - FastAPI `uvicorn main:app --reload --port 8081`

Terminal 2 - Ngrok `ngrok http 8081`

UPDATE OAUTH REDIRECT URI

Localhost

Google Client:
http://localhost:8081/auth/google/callback

Ngrok Tunnel

Google Client:
https://[NGROK_URL]/auth/google/callback

BEST PRACTICES & LIMITATIONS



Best Practices

- **CSRF Protection:** Always validate the state parameter to prevent Cross-Site Request Forgery.
- **Optimize Scopes:** Request only essential scopes (openid, email, profile) to protect user privacy.

><



Limitations

- **Authorization Gap:** OAuth 2.0 only handles Authentication (verifying identity). Developers must manually implement Authorization (roles & permissions) within FastAPI.
- **Platform Dependency:** Losing access to the Google account means losing access to the application.


Thank
you!